

Project: Hybrid Greenhouse Scenario Planner

Name	Mike Kinyanjui Karanja
Student ID	225603898
Date	01/09/2026

Tier Claimed: Level 3

Git Hub Link	https://github.com/CommanderMike/SIT315-Greenhouse-Planner
--------------	---

1. Introduction

Task Overview

I built a C++ program that takes soil moisture, temperature and humidity readings and compares four watering plans. It simulates how each plan might perform under different conditions, then chooses the plan with the lowest combined risk and watering-cost score. The word hybrid in my title refers to MPI and OpenMP working together: MPI shares jobs between the two VMs, and OpenMP splits the calculation between threads inside each VM. Each set of readings is called a sensor window in the program.

I took the idea from my earlier SIT111 smart-plant monitor, which displayed readings and accepted serial commands. Here, I wanted to take that idea further by comparing possible watering decisions. I could not recover the old hardware source, so I built a new software program using synthetic readings. This version evaluates watering plans without connecting to the Arduino or controlling a pump.

The question I wanted to answer was simple: what happens if some jobs take much longer than others? My earlier MPI and hybrid matrix tasks used fixed work assignments. For this project, I

added a shared atomic counter so that a rank can claim another chunk when it finishes its current one. I am claiming Level 3 based on this load-balancing method, the hybrid implementation and the repeated correctness and performance tests.

2. Scope, Input and assumptions

Each job starts with soil moisture, temperature and humidity. For the benchmarks, I generated synthetic readings from the job ID, so every execution mode received the same readings and uncertainty values. I also included a six-row CSV to show that the program can read an input file. When more windows are requested, it cycles through those six rows.

The four watering plans add 0, 2, 4 or 6 moisture percentage points every 12 simulated hours. For each simulated hour, the program estimates moisture loss using temperature, humidity, daylight and an uncertainty factor. It adds a penalty when the soil falls below 40 points or rises above 75 points, plus a cost for watering. The plan with the lowest average score across the scenarios is selected.

I chose the model coefficients to create a repeatable programming experiment. They have not been calibrated for a particular crop, and the watering amounts are moisture points rather than litres. The program lets me compare calculation methods, but its recommendations would need real sensor data and validation before being used in a greenhouse.

Input/control	Recorded setting
CSV columns	soil_pct, temperature_c, humidity_pct
Normal workload	8 uncertainty scenarios for every window
Uneven workload	64 scenarios for the first quarter; 8 for the rest
Benchmark horizon	168 hours
Policies per window	4
Measured chunk size	32 windows

3. Program Design and Impementation

All four modes call the same forecast function. This lets me compare how the work is assigned while keeping the actual calculation the same. Rank zero reads and checks the CSV, then broadcasts the readings as doubles. For generated input, each rank creates readings directly

from the job IDs, so there is no large input array to send between the VMs. In the explanations below, a rank means an MPI process and a chunk means a batch of windows.

Mode	Process assignment	Thread assignment
Sequential	All windows on one rank	One thread
MPI	Fixed contiguous ranges	One thread per rank
Hybrid static	Fixed contiguous ranges	Static OpenMP schedule
Hybrid dynamic	Ranks claim chunks from an atomic counter	Dynamic OpenMP schedule within each chunk

For the dynamic mode, I use `MPI_Win_allocate` to make a counter on rank zero available to the other rank. A rank calls `MPI_Fetch_and_op` with `MPI_SUM` to claim a chunk. For example, with chunks of 32, a returned counter value of 64 means that rank owns windows 64 to 95. The update is atomic, so two ranks cannot claim the same chunk. Rank zero also calculates jobs.

I set the counter to zero and synchronise it before any rank starts claiming work. A barrier makes the ranks wait until that setup is complete. `MPI_Win_flush` finishes each claim before the rank uses the returned value. The window stays locked during the work loop and is unlocked and freed afterwards. If the last chunk would go past the requested job count, the program stops it at that count.

I keep every MPI call on the main thread and check that `MPI_THREAD_FUNNELED` is supported. The OpenMP threads calculate their assigned windows and keep their own statistics. I size the result vector before starting a parallel loop, so each thread writes to a separate result position. Once the loop finishes, I combine the thread statistics.

I used static scheduling as the simpler baseline for jobs with similar costs. With dynamic scheduling, a thread can take another window as soon as it finishes its current one. I used chunks of 32 for the benchmark so that ranks did not need to contact the counter for every window. That was a fixed test setting, not a proven best chunk size. Smaller chunks could balance work more closely, while larger chunks could reduce communication.

After calculation, I gather the result records on rank zero. I define their MPI datatype using the field addresses and resize it to match the C++ structure size, including any padding. Rank zero then sorts the results by ID and compares every selected plan, scenario count, score and watering amount against the sequential reference. A missing or repeated ID also fails verification.

4. Research and the working prototype

The new part I needed to research was how to let ranks claim work safely. I used the MPI Forum material to check atomic updates and window synchronisation [1, 2]. I also checked Open MPI's thread-support documentation [3] and the OpenMP scheduling rules [4]. Together, these helped me move beyond the fixed partition used in my earlier work.

I started with a local CPU build to check the forecast and threading code. I then built the MPI version on both VMs and tested all four modes, first on one VM and then across both. The first cross-VM dynamic run failed at `MPI_Win_allocate`. After selecting Open MPI's `pt2pt` OSC backend, all four modes passed. I included that setting in the run commands so the same setup can be reproduced [5].

The runs also printed graphical authorization warnings. Those lines were still present when verification passed, so I treated them separately from errors that stopped MPI. I kept the warnings visible in the evidence instead of hiding the terminal output.

5. Machines and Build Config

Item	Physical host	Each virtual machine
CPU	Intel Core i5-8265U; reported 1.60 GHz	Same exposed CPU model
CPU resources	4 physical cores; 8 logical processors	2 exposed vCPUs
RAM	Approximately 7.8 GiB reported by PowerShell	Approximately 1.9 GiB reported by <code>free -h</code>
Operating system	Windows 11 Pro, 10.0.26200	Ubuntu 24.04.4 LTS
Guest swap	Not applicable	0 bytes

Although the VM names still say "Ubuntu 22.04", `/etc/os-release` showed Ubuntu 24.04.4 on both. The master used 192.168.20.7 and the worker used 192.168.20.8, with `enp0s3` as the network interface. The master had Open MPI 4.1.6 and g++ 13.3.0. Both VMs ran on my laptop, so the results show execution across two VMs sharing one physical machine.

I used one MPI rank per VM. MPI-only runs had one thread per rank, and hybrid runs had two. The `--bind-to none` option allowed the hybrid processes to use the guest's two vCPUs. Every mode was built with the same C++17, `-O2` and `-fopenmp` flags.

6. Test methods and correctness evidence

6.1 Recorded timings

Before the main benchmark, I tried 10,000 uneven windows with a 168-hour horizon. That run took 3.718 seconds, which gave me a starting point for choosing the test sizes. With about 1.9 GiB RAM and two vCPUs per VM, I settled on 2,000, 10,000 and 30,000 windows. This kept full verification practical and let me increase the workload without making the experiment too long. The smallest cases later proved sensitive to timing overhead, so I use the larger cases for the main scheduling conclusion.

I ran each of the four modes three times at each size, for both uniform and uneven input. That gave 72 measured runs. The script changed the mode order between repeats and left out one small warm-up run. I kept the chunk size, horizon, generated inputs and compiler flags the same. Each measured run also checked its full results after timing had finished.

The reported pipeline time covers calculation, scheduling setup, reductions, rank diagnostics, result gathering and the final completion barrier. I take the longest elapsed time across the ranks, measured before the final timing reduction. Input loading and broadcast, process startup, printing, result sorting and the sequential reference check are outside that timer. To calculate speedup, I divide the separate sequential pipeline median by the parallel pipeline median.

All 72 measured runs passed full verification. Runs with the same inputs also produced the same checksum in every mode. The rank and thread counts added up to every requested window and the expected forecast steps, and each distributed log named both hosts. The small CSV test passed in all four modes too. It used 65 windows, so it also checked a final chunk with just one window.

The reference check uses the same forecast function as the parallel modes. It tells me whether splitting the work changes the results, but it does not prove that the greenhouse model is scientifically correct. The earlier local checks also covered invalid arguments, a NaN reading and a simple moist-soil case where the selected plan was no watering.

```
Oct 7 03:44
micky@sit315-master: ~/M4T1D_Greenhouse
micky@sit315-master:~/M4T1D_Greenhouse$ grep -E '^Rank |Verification|Result checksum:' results/smoke_hybrid-dynamic.txt
Verification: PASSED (all windows)
Result checksum: 4464113005703204511
Rank 0 host sit315-master windows 57 steps 270336 work_seconds 0.052573 thread_windows 40 17
Rank 1 host sit315-worker windows 8 steps 12288 work_seconds 0.055173 thread_windows 4 4
micky@sit315-master:~/M4T1D_Greenhouse$ timeout --kill-after=5s 60s ./greenhouse --node sequential --jobs 10000 --horizon 168 --workload skewed | tee results/calibration_sequential_ske
wed.txt
Starting sequential: 10000 windows, 1 ranks, 1 threads/rank
Mode: sequential
Workload: skewed
Input: deterministic synthetic sensor windows
Windows: 10000
MPI ranks: 1
Threads per rank: 1
Chunk size: 32
Forecast horizon hours: 168
Forecast steps: 147840000
Pipeline seconds: 3.718087
Sequential reference seconds: 2.841427
Verification: PASSED (all windows)
Result checksum: 12758451069099576266
Mean risk score: 65.210110
Mean irrigation points: 74.426800
Plan 0 windows: 0
Plan 1 windows: 458
Plan 2 windows: 2583
Plan 3 windows: 7039
Per-rank workload (work seconds include queue setup for dynamic mode):
Rank 0 host sit315-master windows 10000 steps 147840000 work_seconds 2.450158 thread_windows 10000
First five recommendations (plan, mean score, irrigation points):
Window 0: 3 17.718847 84.000000
Window 1: 3 19.867105 84.000000
Window 2: 2 11.269989 56.000000
Window 3: 2 11.200164 56.000000
Window 4: 2 12.674816 56.000000
micky@sit315-master:~/M4T1D_Greenhouse$
```

Figure 1. The two-VM smoke test and the verified 10,000-window trial used to choose the benchmark sizes. The original screenshot is included in the evidence folder.

7. Performance Evaluation

The tables show the median pipeline time in seconds, using three runs for each case. The last column shows the fastest mode and its speedup over sequential execution. Figure 2 shows the same results as bar charts, with whiskers for the shortest and longest times.

The charts were prepared with Python’s Matplotlib using results/bench_20261007_034644/summary.csv. The C++ program produced the timing logs, tools/summarise.py calculated the medians and ranges, and tools/plot_results.py drew the chart. Python is used here for presenting the results; the scenario calculation is still implemented in C++. The plotting script is included so the figure can be reproduced.

Uniform Workload

Windows	Sequential	MPI	Hybrid static	Hybrid dynamic	Fastest
2,000	0.196	0.107	0.076	0.186	Hybrid static (2.59x)
10,000	0.792	0.400	0.241	0.530	Hybrid static (3.29x)
30,000	2.992	1.272	0.679	2.208	Hybrid static (4.40x)

Uneven Workload

Windows	Sequential	MPI	Hybrid static	Hybrid dynamic	Fastest
2,000	0.742	0.443	0.495	0.631	MPI (1.68x)
10,000	4.242	4.125	2.790	2.679	Hybrid dynamic (1.58x)
30,000	8.534	8.714	7.388	4.611	Hybrid dynamic (1.85x)

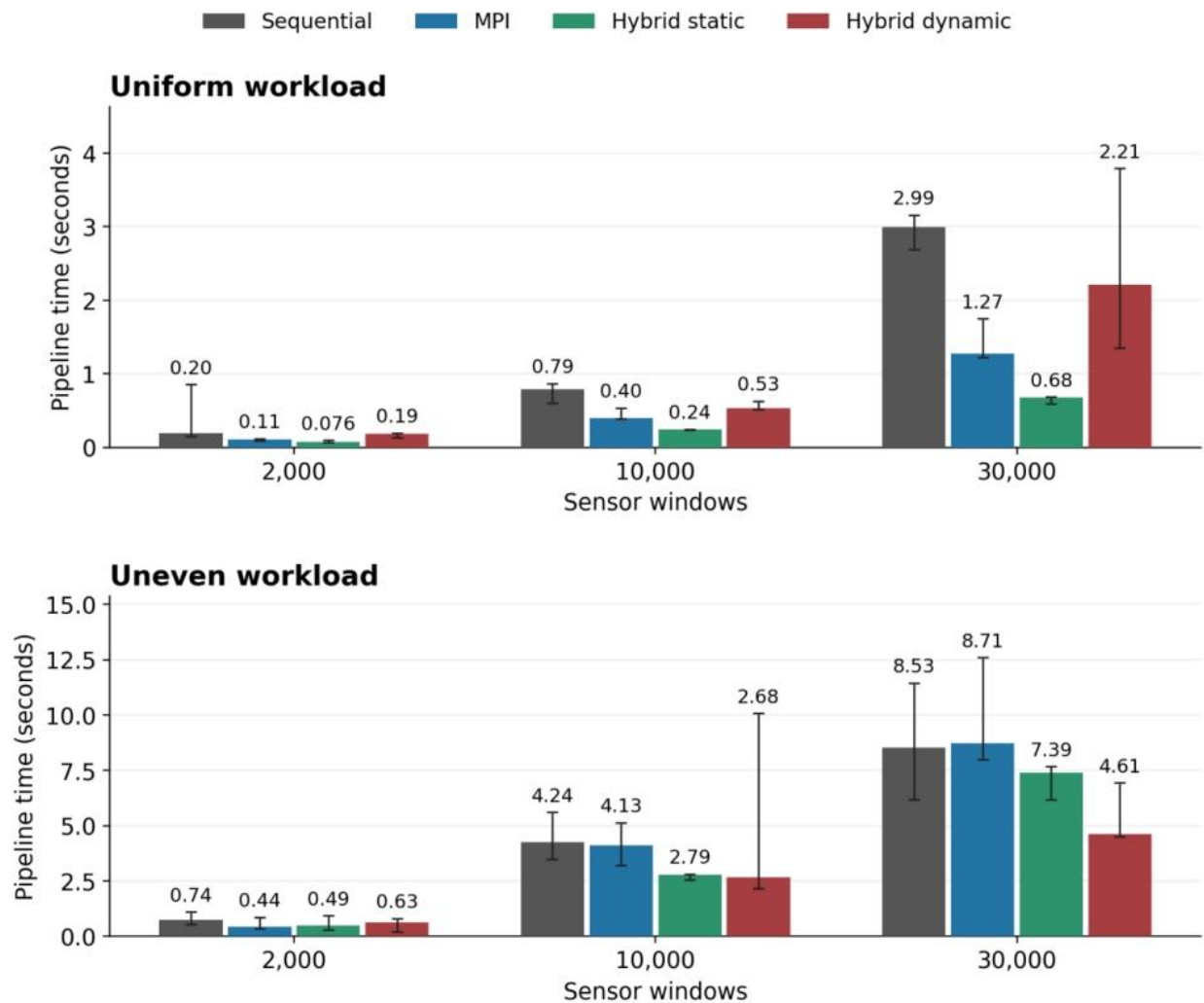


Figure 2. Median pipeline times from three runs per case. Bar heights and labels show medians; whiskers show the minimum and maximum. Both axes start at zero, with a different time range for each workload.

Hybrid static was fastest at all three uniform sizes. At 30,000 windows, it took a median of 0.679 seconds compared with 2.992 seconds for sequential execution, giving a measured speedup of

4.40x. My interpretation is that similar-cost jobs suit a fixed split: the dynamic mode adds counter requests and repeated small-chunk thread work without much imbalance to fix.

The 4.40x result is slightly higher than the four hybrid worker threads, so I would be careful about calling it ideal scaling. The sequential and parallel measurements were separate runs on two VMs sharing the same laptop. System load and scheduling could have affected the ratio, but I did not measure those effects separately.

The uneven workload gave a different result. MPI-only was fastest at 2,000 windows, but hybrid dynamic was fastest at 10,000 and 30,000. For 30,000 windows, dynamic took 4.611 seconds compared with 8.534 seconds sequentially, giving a 1.85x speedup. Its median time was also about 37.6% lower than hybrid static's 7.388 seconds. MPI-only took 8.714 seconds, which was slightly slower than sequential execution.

This is why I would choose the scheduling method based on the workload. Dynamic was useful for the larger uneven cases, but it took 2.208 seconds on the largest uniform case, well behind hybrid static. The extra scheduling is worthwhile when it saves more waiting time than it adds in overhead.

What my workload counts explain

The rank counts help explain this difference. In the 30,000-window uneven test, the fixed split gave each rank 15,000 windows. However, all the expensive windows were in the master's range. The master performed 362.880 million forecast steps, while the worker performed 80.640 million. The job counts looked equal, but the calculation was split 4.5 to 1.

Mode / rank	Windows	Forecast steps (millions)	Rank work elapsed (s)
Hybrid static / master	15,000	362.880	7.283
Hybrid static / worker	15,000	80.640	0.786
Hybrid dynamic / master	22,480	324.366	4.539
Hybrid dynamic / worker	7,520	119.154	4.548

I took these rows from the run with the median pipeline time for each mode. With dynamic claiming, the worker handled more forecast steps and the two ranks finished with similar work elapsed times. They still processed different numbers of windows, which is expected when jobs have different costs. The dynamic work timer includes queue setup and waiting for claims, so I have not treated it as CPU utilisation.

Variation and limits of the comparison

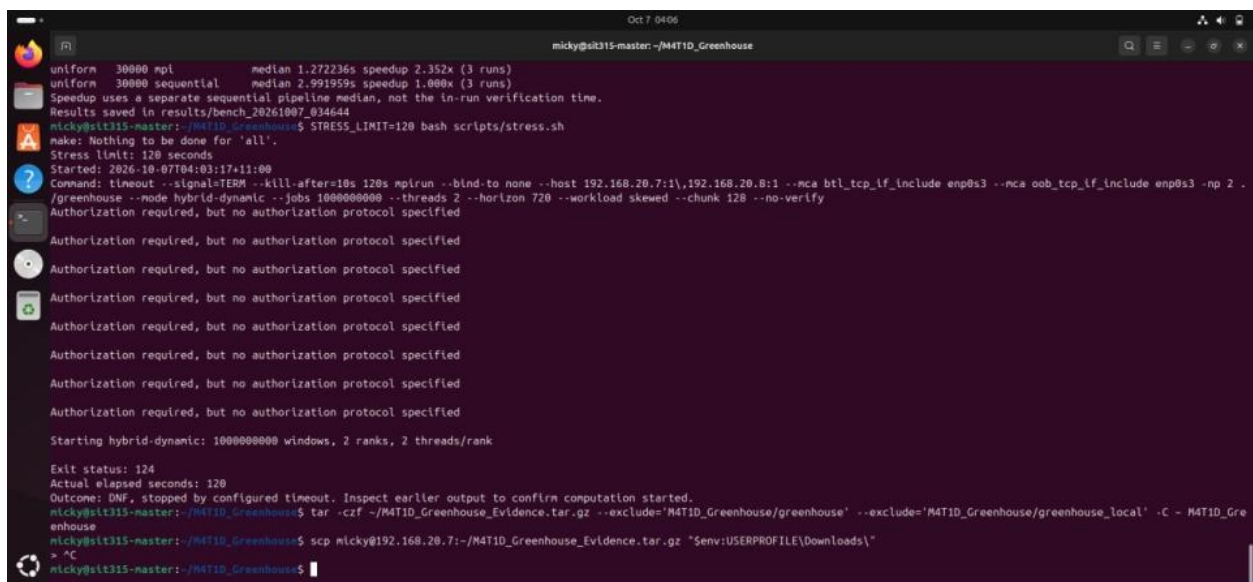
Some timings varied quite a lot. The largest uneven dynamic runs ranged from 4.522 to 6.941 seconds. At 10,000 windows, the dynamic times were 10.069, 2.159 and 2.679 seconds. I kept the slow run in the results. Using the median limits its effect on the comparison, but three repeats are not enough to explain every delay or give a precise confidence interval.

The smallest uniform hybrid-static runs were under 0.1 seconds, where overhead and small timing differences matter more. I therefore put more weight on the larger cases when discussing scheduling. Testing on separate physical machines and using more repeats would help show how well these results hold beyond my laptop.

8. Controlled DNF Stress test

For the stress test, I requested one billion windows with a 720-hour horizon and uneven scenario costs. I used two MPI ranks, two threads per rank and chunks of 128. I also used `--no-verify` so the program would not try to store a billion result records. This let me test a very long calculation with a fixed time limit.

The program printed its hybrid-dynamic startup message, but did not finish within the 120-second limit. The timeout stopped it with exit status 124, and the log recorded 120 seconds elapsed. I recorded this as DNF due to timeout. There was no final result or reported out-of-memory error, so I cannot say how many of the billion windows were completed.

A terminal window titled 'micky@ul315-master: ~/MAT1D_Greenhouse' showing the output of a stress test. The output includes performance metrics for 30000 MPI and sequential runs, a command to run a stress test with a 120-second timeout, and a series of 'Authorization required' messages. The test is then started with hybrid-dynamic scheduling, and the output shows the exit status 124 and the elapsed time of 120 seconds. The final output shows the test was stopped by a configured timeout.

```
Oct 7 04:06
micky@ul315-master: ~/MAT1D_Greenhouse

uniform 30000 mpi          median 1.272236s speedup 2.352x (3 runs)
uniform 30000 sequential  median 2.991959s speedup 1.000x (3 runs)
Speedup uses a separate sequential pipeline median, not the in-run verification time.
Results saved in results/bench_20261007_034644
micky@ul315-master: ~/MAT1D_Greenhouse$ STRESS_LIMIT=120 bash scripts/stress.sh
make: Nothing to be done for 'all'.
Stress limit: 120 seconds
Started: 2026-10-07T04:03:17+11:00
Command: timeout --signal=TERM --kill-after=10s 120s mpirun --bind-to none --host 192.168.20.7:1,192.168.20.8:1 --mca btl_tcp_if_include enp0s3 --mca oob_tcp_if_include enp0s3 -np 2 .
/greenhouse --mode hybrid-dynamic --jobs 1000000000 --threads 2 --horizon 720 --workload skewed --chunk 128 --no-verify
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Starting hybrid-dynamic: 1000000000 windows, 2 ranks, 2 threads/rank
Exit status: 124
Actual elapsed seconds: 120
Outcome: DNF, stopped by configured timeout. Inspect earlier output to confirm computation started.
micky@ul315-master: ~/MAT1D_Greenhouse$ tar -czf ~/MAT1D_Greenhouse_Evidence.tar.gz --exclude='MAT1D_Greenhouse/greenhouse' --exclude='MAT1D_Greenhouse/greenhouse_local' -C ~/MAT1D_Greenhouse
micky@ul315-master: ~/MAT1D_Greenhouse$ scp micky@192.168.20.7:~/MAT1D_Greenhouse_Evidence.tar.gz "$env:USERPROFILE\Downloads\"
> ^C
micky@ul315-master: ~/MAT1D_Greenhouse$
```

Figure 3. The saved stress-test output records the requested workload, startup message, exit status 124 and 120-second elapsed duration.

9. How to reproduce the project (All files are available on the repo)

Build the project on both VMs, with the same folder path on each. Then run the commands below on the master. The worker needs to be running and reachable through SSH.

```
cd ~/M4T1D_Greenhouse
make
export MPI_HOSTS=192.168.20.7:1,192.168.20.8:1
export MPI_RANKS=2 MPI_IFACE=enp0s3 OMP_THREADS=2
export OMPI_MCA_osc=pt2pt
```

The exact compile command used by the Makefile is:

```
mpicxx -O2 -std=c++17 -fopenmp -Wall -Wextra -Wpedantic \
src/greenhouse.cpp -o greenhouse
```

Confirm both VM hostnames and then run the verified four-mode test:

```
timeout --kill-after=5s 30s mpirun --host "$MPI_HOSTS" \
--bind-to none -np 2 --mca btl_tcp_if_include "$MPI_IFACE" \
--mca oob_tcp_if_include "$MPI_IFACE" hostname
timeout --kill-after=5s 45s bash scripts/smoke_test.sh
```

A direct hybrid demonstration using the supplied CSV is:

```
mpirun --host "$MPI_HOSTS" --bind-to none -np 2 \
--mca osc pt2pt --mca btl_tcp_if_include "$MPI_IFACE" \
--mca oob_tcp_if_include "$MPI_IFACE" ./greenhouse \
--mode hybrid-dynamic --jobs 65 --threads 2 --chunk 8 \
--workload skewed --input data/example_readings.csv
```

Reproduce the measured benchmark and controlled stress case with:

```
SIZES="2000 10000 30000" REPEATS=3 HORIZON=168 RUN_LIMIT=90 \
timeout --kill-after=10s 20m bash scripts/benchmark.sh
STRESS_LIMIT=120 bash scripts/stress.sh
```

My Git repo package contains the source, Makefile, test scripts, sample CSV, original logs, summaries and screenshots. The README explains the working configuration. On another network, the IP addresses and interface name will need to be changed. The pt2pt setting is specific to the recorded Open MPI 4.1 setup. The submitted greenhouse.cpp contains the complete application. The comments were expanded after testing; every non-comment source line still matches the benchmarked version.

To recreate Figure 2, use Python with Matplotlib installed and run:

```
python3 tools/plot_results.py results/bench_20261007_034644/summary.csv
```

If only greenhouse.cpp is downloaded from OnTrack, compile it directly and run a small generated-input check with:

```
mpicxx -O2 -std=c++17 -fopenmp -Wall -Wextra -Wpedantic greenhouse.cpp -o greenhouse  
./greenhouse --mode sequential --jobs 65 --horizon 48 --workload skewed
```

10. Conclusion and next improvements

The main thing I learnt from this project is that splitting the same number of jobs between ranks does not always give them the same amount of work. Static hybrid scheduling worked best when the window costs were similar. For the larger uneven cases, the shared counter let ranks keep taking work and reduced the waiting caused by a fixed split. The verification checks showed that both methods kept the calculation results consistent.

If I continued this project, I would compare chunk sizes and test on separate physical machines. I would also need to calibrate the forecast with real sensor readings before connecting it to hardware. For now, I have focused on the working software, the added load-balancing method and results that I can explain from the saved evidence.

References

[1] MPI Forum. MPI 4.1, section 13.3.4.3: Fetch and Op.

<https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report/node320.htm>

[2] MPI Forum. MPI 5.0, section 13.8: RMA synchronisation examples.

<https://www.mpi-forum.org/docs/mpi-5.0/mpi50-report/node344.htm>

[3] Open MPI. MPI_Init_thread manual, version 4.1.

https://www.open-mpi.org/doc/v4.1/man3/MPI_Init_thread.3.php

[4] OpenMP ARB. OpenMP 5.1, section 2.11.4: Worksharing-loop construct.

<https://www.openmp.org/spec-html/5.1/openmpsu48.html>

[5] Open MPI. Modular Component Architecture: runtime parameter settings.

<https://docs.open-mpi.org/en/main/mca.html>